WEEK 3 INTERNSHIP REPORT

SQL & MICROSOFT EXCEL

Task 1: SELECT Statements and Column Aliases

Aim: Create a table, insert sample student records and retrieve all or selected columns.

Description: SELECT retrieves rows from a table. AS gives a temporary display name to a column.

Program:

```
CREATE TABLE Students (
  StudentID INT PRIMARY KEY,
  StudentName VARCHAR(30),
  Department VARCHAR(20),
  Marks INT,
  Salary INT
);

INSERT INTO Students VALUES
(101,'Ravi','IT',85,40000),
(102,'Priya','HR',90,35000),
(103,'Kiran','IT',72,40000),
(104,'Sita','Sales',84,55000),
(105,'Arun','Sales',82,50000);

SELECT * FROM Students;
SELECT StudentName AS Name, Marks AS Score
FROM Students;
```

Result:

```
StudentID  StudentName  Department  Marks  Salary
101        Ravi         IT          85     40000
102        Priya        HR          90     35000
103        Kiran        IT          72     40000
104        Sita         Sales       84     55000
105        Arun         Sales       82     50000

Name   Score
Ravi   85
Priya  90
Kiran  72
Sita   84
Arun   82
```

```
+-----------+-------------+------------+-------+
| StudentID | StudentName | Department | Marks |
+-----------+-------------+------------+-------+
|         1 | Ravi        | CSE        |    85 |
|         2 | Priya       | ECE        |    90 |
|         3 | Kiran       | CSE        |    78 |
|         4 | Sita        | IT         |    88 |
|         5 | Arun        | ECE        |    72 |
+-----------+-------------+------------+-------+
```

```
+-------------+-------+
| StudentName | Marks |
+-------------+-------+
| Ravi        |    85 |
| Priya       |    90 |
| Kiran       |    78 |
| Sita        |    88 |
| Arun        |    72 |
+-------------+-------+
```

```
+-------+------------+
| Name  | TotalMarks |
+-------+------------+
| Ravi  |         85 |
| Priya |         90 |
| Kiran |         78 |
| Sita  |         88 |
| Arun  |         72 |
+-------+------------+
```

Conclusion: SELECT and aliases are basic SQL tools for displaying the required data clearly.

Task 2: WHERE, ORDER BY and Aggregate Functions

Aim: Filter records, sort results and calculate summary values.

Description: WHERE filters rows; ORDER BY sorts them; COUNT, SUM, AVG, MIN and MAX calculate aggregate results.

Program :

```
SELECT * FROM Students WHERE Marks >= 80;
SELECT * FROM Students ORDER BY Marks DESC;
SELECT COUNT(*) AS TotalStudents,
       SUM(Marks) AS TotalMarks,
       AVG(Marks) AS AverageMarks,
       MIN(Marks) AS MinimumMarks,
       MAX(Marks) AS MaximumMarks
FROM Students;
```

Result:

```
Marks >= 80: Ravi (85), Priya (90), Sita (84), Arun (82)
```

ORDER BY Marks DESC: Priya 90, Ravi 85, Sita 84, Arun 82, Kiran 72

```
TotalStudents  TotalMarks  AverageMarks  MinimumMarks  MaximumMarks
5              413         82.6          72            90
```

StudentID	StudentName	Department	Marks
1	Ravi	CSE	85
2	Priya	ECE	90
4	Sita	IT	88

StudentID	StudentName	Department	Marks
2	Priya	ECE	90
4	Sita	IT	88
1	Ravi	CSE	85
3	Kiran	CSE	78
5	Arun	ECE	72

TotalStudents
5

TotalMarks
413

AverageMarks
82.6000

MinimumMarks
72

```
+--------------+
| MaximumMarks |
+--------------+
|           90 |
+--------------+
```

Conclusion: Filtering, sorting and aggregate functions quickly summarize a dataset.

Task 3: GROUP BY and HAVING

Aim: Summarize marks by department and filter grouped results.

***Description*:** GROUP BY forms groups based on a column. HAVING applies a condition to aggregate/group results.

Program :

```
SELECT Department, COUNT(*) AS EmployeeCount,
       AVG(Marks) AS AverageMarks
FROM Students
GROUP BY Department;

SELECT Department, AVG(Marks) AS AverageMarks
FROM Students
GROUP BY Department
HAVING AVG(Marks) >= 80
```

Result:

```
Department  EmployeeCount  AverageMarks
HR          1              90.0
IT          2              78.5
Sales       2              83.0

HAVING AVG(Marks) >= 80:
HR          90.0
Sales       83.0
```

```
+------------+---------------+
| Department | TotalStudents |
+------------+---------------+
| CSE        |             2 |
| ECE        |             2 |
| IT         |             1 |
+------------+---------------+
+------------+--------------+
| Department | AverageMarks |
+------------+--------------+
| CSE        |      81.5000 |
| ECE        |      81.0000 |
| IT         |      88.0000 |
+------------+--------------+
+------------+--------------+
| Department | AverageMarks |
+------------+--------------+
| CSE        |      81.5000 |
| ECE        |      81.0000 |
| IT         |      88.0000 |
+------------+--------------+
```

Conclusion: GROUP BY provides category-wise summaries, while HAVING filters those summaries.

Task 4: SQL Joins

Aim: Combine records from two related tables.

Description: INNER JOIN returns matching rows; LEFT JOIN keeps all rows from the left table and matching rows from the right table.

Program:

```
CREATE TABLE Departments (
  Department VARCHAR(20) PRIMARY KEY,
  Manager VARCHAR(30)
);
INSERT INTO Departments VALUES
('IT','Meera'),('HR','Ramesh'),('Sales','Anil');

SELECT s.StudentName, s.Department, d.Manager
FROM Students s
INNER JOIN Departments d
ON s.Department = d.Department;

SELECT s.StudentName, d.Manager
FROM Students s
LEFT JOIN Departments d
ON s.Department = d.Department;
```

Result:

```
INNER JOIN (matching department records)
Ravi   IT     Meera
Priya  HR     Ramesh
Kiran  IT     Meera
Sita   Sales  Anil
Arun   Sales  Anil
```

```
LEFT JOIN returns all Students rows and matching manager values.
+-------------+----------------+
| StudentName | DepartmentName |
+-------------+----------------+
| Kiran       | CSE            |
| Ravi        | CSE            |
| Priya       | ECE            |
| Sita        | IT             |
+-------------+----------------+
+-------------+----------------+
| StudentName | DepartmentName |
+-------------+----------------+
| Ravi        | CSE            |
| Priya       | ECE            |
| Kiran       | CSE            |
| Sita        | IT             |
+-------------+----------------+
+-------------+----------------+
| StudentName | DepartmentName |
+-------------+----------------+
| Kiran       | CSE            |
| Ravi        | CSE            |
| Priya       | ECE            |
| Sita        | IT             |
```

Conclusion: Joins combine related information using a common column.

Task 5: Subqueries

Aim: Use a query inside another query to compare values with a calculated result.

Description: The first subquery calculates the average mark. The second finds the maximum salary and returns the matching employee.

Program:

```
SELECT StudentName, Marks
FROM Students
WHERE Marks > (SELECT AVG(Marks) FROM Students);

SELECT StudentName, Salary
FROM Students
WHERE Salary = (SELECT MAX(Salary) FROM Students);
```

Result:

```
Average marks = 82.6
StudentName  Marks
Ravi         85
Priya        90
Sita         84

Highest salary = 55000
StudentName  Salary
Sita         55000
```

```
+--------------+--------+
| EmployeeName | Salary |
+--------------+--------+
| Kiran        |  50000 |
| Sita         |  60000 |
+--------------+--------+
+-------------+-------+
| ProductName | Price |
+-------------+-------+
| Laptop      | 40000 |
+-------------+-------+
```

***Conclusion*:** Subqueries allow a query to use the result of another query as a condition.

Task 6: Excel Formatting, Sorting and Filtering

Aim: Format an employee dataset and make records easier to inspect.

Description: Convert the range to a formatted table, sort salary in descending order and filter the Department column.

Program:

```
Sample Excel table (A1:C7)
Employee | Department | Salary
Ravi     | IT         | 40000
Priya    | HR         | 35000
Kiran    | IT         | 40000
Sita     | Sales      | 55000
Arun     | Sales      | 50000
Meena    | HR         | 40000

Procedure:
1. Select the data range including headings.
2. Home > Format as Table; choose a style.
3. Data > Sort: sort Salary from Largest to Smallest.
4. Data > Filter: use Department dropdown to show IT only.
```

Result:

```
Sorted by Salary (Largest to Smallest)
Sita   Sales  55000
Arun   Sales  50000
Ravi   IT     40000
Kiran  IT     40000
Meena  HR     40000
Priya  HR     35000

Filter Department = IT
Ravi   IT  40000
Kiran  IT  40000
```

Employee Name	Department	Salary
Sita	Sales	60000
Kiran	IT	50000
Divya	Sales	45000
Priya	HR	40000
Arun	HR	35000
Ravi	IT	30000

Employee ID	Employee Name	Department	Salary
101	Ravi	IT	30000
103	Kiran	IT	50000

Conclusion: Formatting improves readability; sorting orders records and filters display only matching records.

Task 7: Excel Formulas and IF Function

Aim: Calculate total, average, minimum, maximum and count, then classify salaries.

Description: Enter the formulas using the salary range C2:C7. Use IF to return a status based on a salary threshold.

Program:

```
Assume salaries are in C2:C7.
Total salary:       =SUM(C2:C7)
Average salary:     =AVERAGE(C2:C7)
Minimum salary:     =MIN(C2:C7)
Maximum salary:     =MAX(C2:C7)
Number of salaries: =COUNT(C2:C7)
```

```
In D2, enter and fill down:
=IF(C2>=40000,"Eligible","Review")
```

Function		Result
Total Salary		₹ 260,000.00
Average Salary		₹ 43,333.33
Minimum Salary		₹ 30,000.00
Maximum Salary		₹ 60,000.00
Number of Employees		₹ 6.00

Result:

```
SUM       260000
AVERAGE   43333.33
MIN       35000
MAX       55000
COUNT     6

IF result by employee (salary >= 40000):
Ravi  Eligible
Priya Review
Kiran Eligible
Sita  Eligible
Arun  Eligible
Meena Eligible
```

Conclusion: Excel functions automate calculations and the IF function applies a logical condition to each row.

Task 8: VLOOKUP and XLOOKUP

Aim: Retrieve department names from a reference table using employee department IDs.

Description: VLOOKUP searches the first column of the lookup range. XLOOKUP uses separate lookup and return ranges and can show a fallback message.

Program:

```
Lookup table (I2:J4)
Department ID | Department Name
1             | IT
2             | HR
3             | Sales

Assume employee Department ID is in C2.
VLOOKUP (D2):
=VLOOKUP(C2,$I$2:$J$4,2,FALSE)

XLOOKUP (E2):
=XLOOKUP(C2,$I$2:$I$4,$J$2:$J$4,"Not found")
```

Result:

Employee Name	Department ID	VLOOKUP Result	XLOOKUP Result
Ravi	1	IT	IT
Priya	2	HR	HR
Kiran	1	IT	IT
Sita	3	Sales	Sales
Arun	2	HR	HR

```
VLOOKUP uses a column index; XLOOKUP separately specifies lookup and return ranges.
```

***Conclusion*:** Lookup functions connect related tables and reduce manual data entry. XLOOKUP is available in supported newer Excel versions.

Task 9: PivotTable and Chart

Aim: Summarize salary totals by department and present the comparison visually.

Description: Create a PivotTable, place Department in Rows and Salary in Values, then insert a PivotChart. The displayed figures are the expected summary for the sample data.

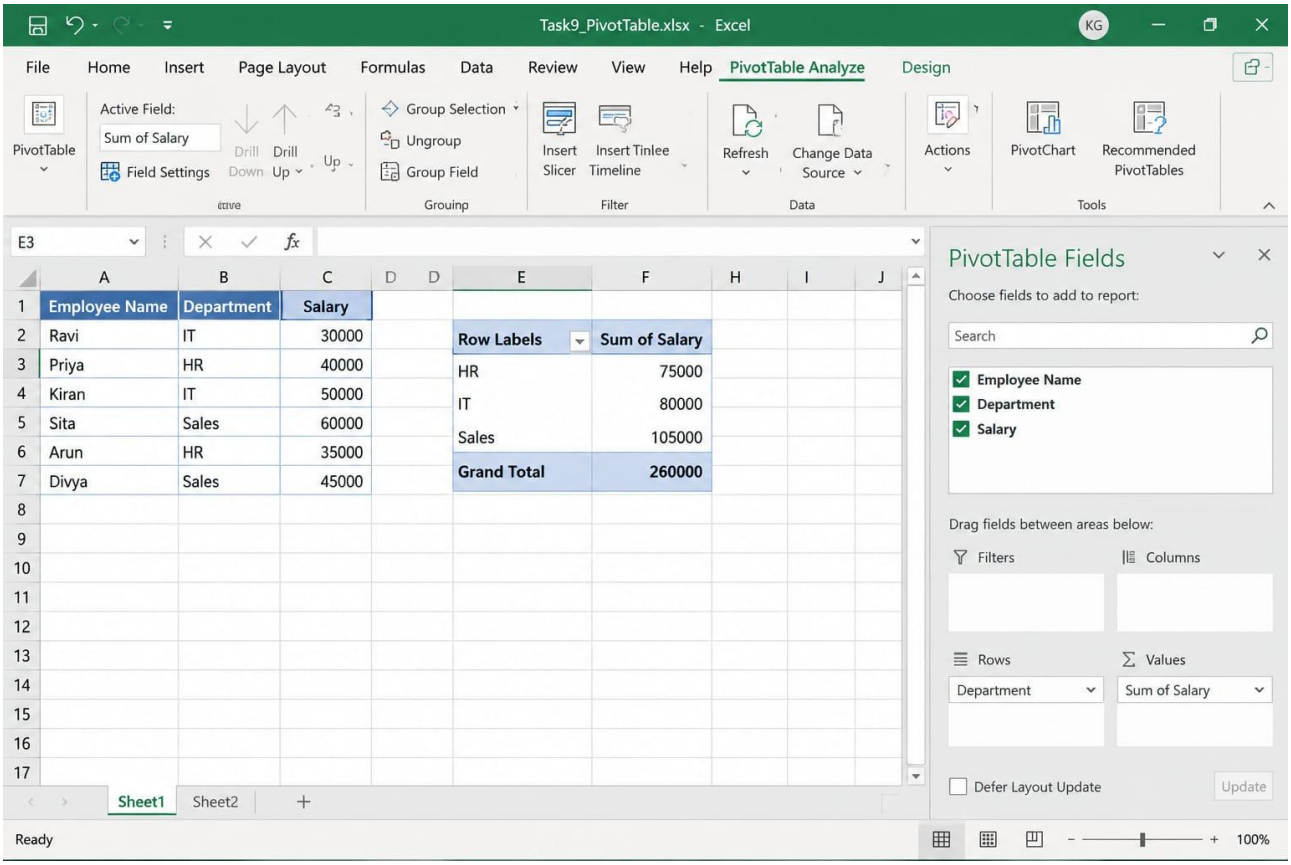
Program:

```
Using the employee table (A1:C7):
1. Select the table including headings.
2. Insert > PivotTable > New Worksheet.
3. Drag Department to Rows.
4. Drag Salary to Values; ensure it is set to Sum.
5. Insert > PivotChart; choose a column chart.
6. Add a clear title, e.g. Total Salary by Department.

If using formulas to verify totals:
=SUMIF(B2:B7,"IT",C2:C7)
=SUMIF(B2:B7,"HR",C2:C7)
=SUMIF(B2:B7,"Sales",C2:C7)
```

Result:

```
PivotTable: Sum of Salary by Department
Department  Sum of Salary
HR          75000
IT          80000
Sales       105000
Grand Total 260000
```



Conclusion: PivotTables summarize large tables quickly, and charts make comparisons easier to understand.